A claim algebra for agent networks

The verifiable inter-agent message as a typed mathematical object — a category-theoretic abstraction, its algebraic realization, and the code.

Working note · companion to the focused-harness actor-model design and the Credit & Loan Agreement Workbench · math via MathJax, diagrams via Mermaid + SVG

A network of LLM agents is only as trustworthy as the messages they pass each other. Today those messages are unverifiable prose, so a confident error at one hop propagates with full authority to the next. This note proposes the fix as a single mathematical object — a *claim* that carries its own provenance, a computed confidence, and a fail-closed semantics — and argues that the object is best understood as a three-level tower: a categorical abstraction, realized by concrete algebraic structures, implemented as a programming type.

> **The one-paragraph thesis.** Make every inter-agent message a value annotated by an element of a commutative semiring $(K, \oplus, \otimes, 0, 1)$, with the distinguished element `UNRESOLVED` as the zero 0. Then *fail-closed* is the annihilator law $0 \otimes x = 0$ — a theorem, not a discipline a node can forget; *pluggable trust models* are semiring homomorphisms out of the free provenance semiring; and the message is a programming type: a semiring-graded applicative refined by a decidable verification predicate. The mathematics is decades old; using it as the *typed wire format between LLM agents* is the new part.

How this document is organized

1. The idea — what a claim is, and why prose is the wrong wire format.
2. The tower — why this is a category-theoretic abstraction realized in algebra realized in a type.
3. The categorical layer ABSTRACTION — monoidal categories, lax monoidal functors, the free-object universal property.
4. The algebraic layer REALIZATION — the semiring, the provenance polynomials, the confidence lattice, the properties.
5. The type layer CODE — `Claim[K, A]` as a graded validation applicative plus a verification refinement.
6. The workbench is the N = 1 instance — the object we have already built.

7. A second instance: cost routing REALIZATION — an agent fulfillment network as the tropical-semiring sibling.
8. A third instance: adversarial games REALIZATION — Diplomacy as the trust-semiring sibling, where the algebra meets its boundary.
9. Where the algebra stops — the honest limits, which double as the roadmap.
10. References.

1 · The idea

Picture three agents. One extracts a borrower's total debt from a filing; one extracts its EBITDA; a third divides them to report leverage. In a prose pipeline, the third agent receives "*debt is $1.2bn*" and "*EBITDA is $400m*" as sentences, believes them, and emits "*leverage is 3.0x*" as another sentence. Nothing in the message records *where* $1.2bn came from, *how sure* the extractor was, or what should happen if the EBITDA agent actually failed and guessed. An error at hop one reaches the signed memo at hop three wearing the same confident clothes as a fact.

The remedy is to stop passing values and start passing **claims**. A claim bundles six things, and the rest of this note is about giving those six a precise mathematical home.

#	FEATURE OF A CLAIM	INFORMAL MEANING
1	**Value or `UNRESOLVED`**	A payload, or the first-class absence of a trustworthy one.
2	**Provenance**	Where the value came from — source spans, tool outputs, and for derived claims, the input claims it was computed from (its lineage).
3	**Confidence**	A graded signal, *computed from the derivation*, never asserted by the producer.
4	**Fail-closed**	Combine anything conjunctively with an `UNRESOLVED` input and the result is `UNRESOLVED`. Missing support cannot be laundered into a confident answer.
5	**Composition**	Conjunctive ("derived from A *and* B") and disjunctive ("corroborated by A *or* B").
6	**Verification**	The value is checkable against its provenance — a predicate separate from how-confident-given-the-derivation.

2 · The tower: abstraction, realization, implementation

The natural question — and the one that prompted this note — is whether the right description is category-theoretic. The answer is yes, but with discipline: category theory belongs at the *top*, where it captures the abstraction that several concrete structures share, and it should be used only where it earns its keep. Below it sit the algebraic structures that *realize* the abstraction, and below those, the type that *implements* them.

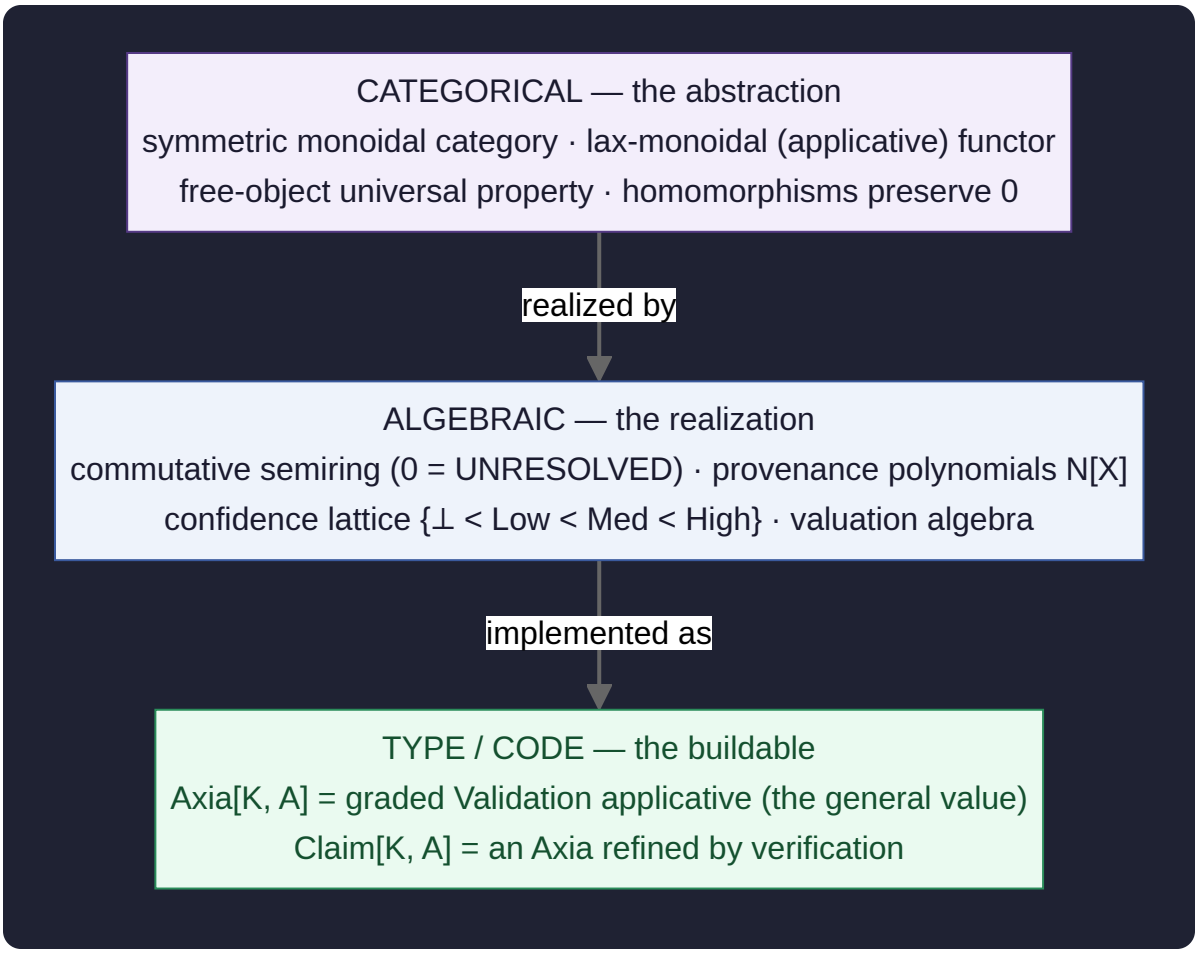

Figure 1 — The three-level tower. Each level is a faithful description of the same object at a different altitude; the arrows are "is realized by" and "is implemented as." The categorical level explains *why* the lower levels are inevitable; the type level is what you compile.

> **The honesty rule for the top level.** Category theory earns its place here in exactly two ways: the *universal property of the free semiring* (which makes "compute provenance once, evaluate many trust models" a one-line theorem) and the *preservation of the zero by morphisms* (which makes fail-closed hold under *every* trust model at once). Past those, further categorical generality would be decoration, and the note drops to the algebra. Abstraction is a tool for compression, not a flex.

> **Two layers, two names.** The general object at the algebraic and type levels — a value (or UNRESOLVED) carried with its provenance and a grade in the semiring K — is an **Axia** (Greek *axía*, "worth"). It is value-agnostic: the payload may be a fact, a cost, or a trust level, and the entire algebra of §3 and §4 operates at this layer. A **Claim** is an Axia refined by a verification predicate — an Axia that *asserts* something and can be checked against its provenance. So a Claim is an Axia that verifies. The framework keeps the name "claim algebra" because verification is its motivating purpose, but the general structure underneath is an Axia: the cost-routing instance (§7) is naturally an Axia, a graded cost; the workbench's cited facts and Diplomacy's promises are Claims, because there the verification is the point.

---

3 · The categorical layer ABSTRACTION

3.1 · Claims compose — so they live in a monoidal category

Agents take claims in and put claims out; they are morphisms. Two claims can be combined into one (the leverage example). The structure that captures "objects, morphisms, and a way to combine things side by side" is a **symmetric monoidal category** $(\mathcal{C}, \otimes, I)$: the tensor $\otimes$ is conjunctive composition, and the unit I is the trivially-true claim. An agent network is then a **string diagram** in $\mathcal{C}$ — boxes are agents, wires carry claims, side-by-side wires are conjunction, and the diagram's value is read by composing the boxes [5, 6, 12].

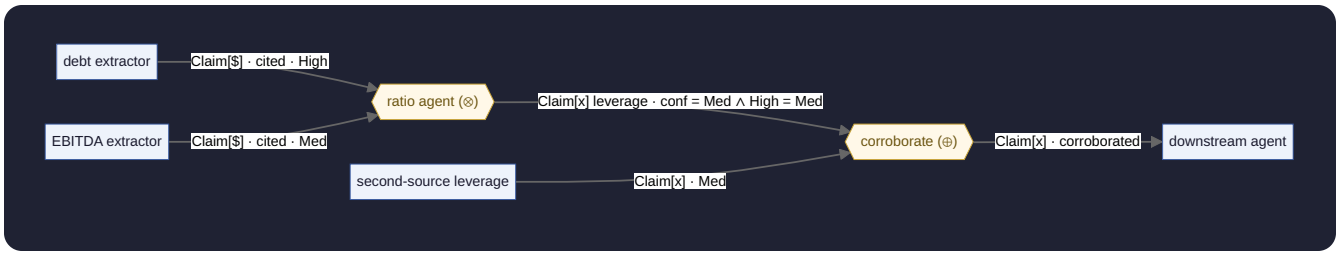

Figure 2 — An agent network as a string diagram. Wires are typed claims, not prose. Side-by-side inputs into a box are a tensor $\otimes$ (conjunction); the $\oplus$ box is disjunctive corroboration. The diagram *is* a morphism in a symmetric monoidal category, which is what makes the network's meaning compositional.

3.2 · The trust annotation is a lax monoidal functor

A claim is a value *decorated* with provenance and confidence. The decoration is a functor F that, crucially, knows how to combine decorations when values combine. The precise statement is that F is a **lax monoidal functor**: there is a natural map

$$Fa \otimes Fb \longrightarrow F(a \otimes b),$$

which says "given a decorated a and a decorated b, produce a decorated $a \otimes b$" — i.e. combine the two annotations into the annotation of the combination. This is not a coincidence of vocabulary: **applicative functors are exactly lax monoidal functors** [2, 11]. The programming type in §5 is the same statement, written in a language with a compiler.

3.3 · The universal property — compute provenance once, evaluate many trust models

Here is where category theory pays for itself. Let X be the set of leaf sources (this clause, that tool output). The provenance of a derived claim is a polynomial in the X: a sum (alternative derivations) of products (conjoined supports). These polynomials form the **free commutative semiring** $\mathbb{N}[X]$ — the free object on X in the category of commutative semirings. Freeness is a *universal property*: for any commutative semiring K and any valuation $\nu : X \to K$ (a base confidence, cost, or trust score for each leaf), there is a **unique semiring homomorphism**

$$\hat{\nu} : \mathbb{N}[X] \longrightarrow K \quad \text{such that} \quad \hat{\nu} \circ \iota = \nu,$$

where $\iota : X \hookrightarrow \mathbb{N}[X]$ embeds the leaves. In words: **you compute the lineage polynomial once, and every concrete trust measure is a homomorphic evaluation of that one polynomial.** This is the Green–Karvounarakis–Tannen provenance-semiring theorem [1], stated as the universal property it is. The square below commutes for every homomorphism — "compose then evaluate" equals "evaluate then compose."

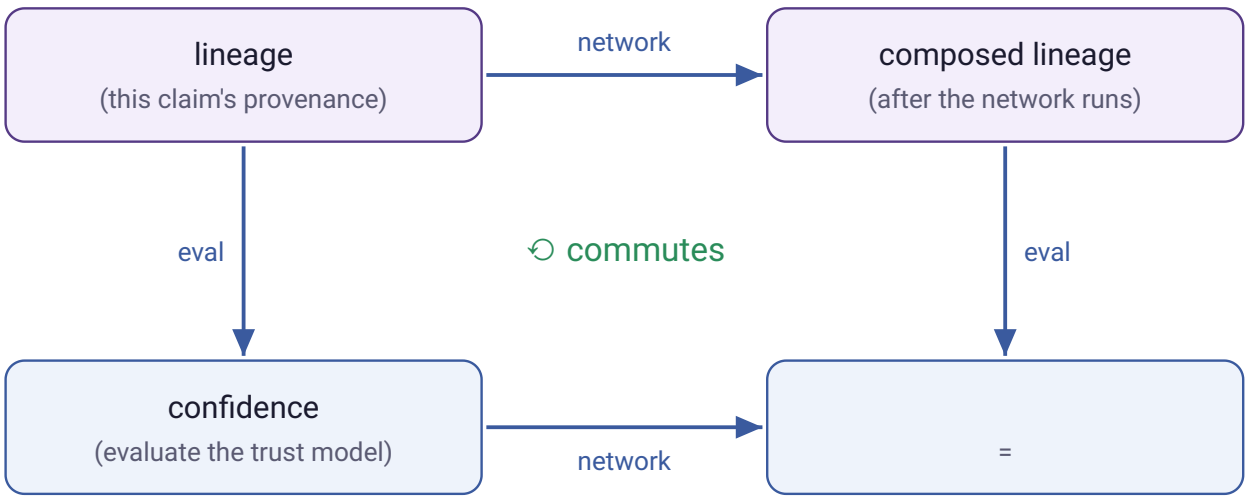

Figure 3 — The homomorphism theorem as a commuting square. Because h is a semiring homomorphism, evaluating the trust model and running the network can be done in either order. You can therefore **swap the trust model without recomputing the network** — re-evaluate the same lineage polynomial under a different h.

Why fail-closed is free, and model-independent. Every semiring homomorphism preserves the zero: $h(0) = 0$. Identify UNRESOLVED with 0; then any conjunctive derivation touching it is 0, and that fact survives *every* evaluation h into *every* trust model K. Fail-closed is not a feature you implement once and hope every agent respects — it is a structural property of the category of semirings.

---

4 · The algebraic layer REALIZATION

4.1 · The object: a commutative semiring with an absorbing zero

A **commutative semiring** $(K, \oplus, \otimes, 0, 1)$ is two commutative monoids on one set — $\oplus$ with identity 0, $\otimes$ with identity 1 — where $\otimes$ distributes over $\oplus$ and 0 annihilates $\otimes$:

$$a \oplus 0 = a, \qquad a \otimes 1 = a, \qquad a \otimes (b \oplus c) = (a \otimes b) \oplus (a \otimes c), \qquad \boxed{0 \otimes a = 0}.$$

The boxed law is the whole safety story. The feature-to-structure map is exact:

CLAIM FEATURE	SEMIRING STRUCTURE
Value or UNRESOLVED	A lifted carrier $A_\perp$; $\perp$ is the zero 0 — "no value" and "no support" are the same element.
Conjunctive composition (AND)	$\otimes$: lineage is the union of inputs; confidence is the weakest link ($\min$) or product; 0 is absorbing.
Disjunctive corroboration (OR)	$\oplus$: lineage is the union of alternative derivations; confidence combines up ($\max$ or noisy-OR); identity is UNRESOLVED.
Verified ground truth	The one 1: composing with a fully-verified citation adds no doubt.
Fail-closed	The annihilator $0 \otimes a = 0$.

4.2 · The universal instance and the concrete instance

The *universal* K is the provenance polynomial semiring $\mathbb{N}[X]$ from §3.3 — you compute in it once. The *concrete* K you evaluate into is a design choice, and several are standard: the Viterbi semiring $([0,1], \max, \times)$ for most-likely-derivation confidence; the fuzzy semiring $([0,1], \max, \min)$ for weakest-link; the tropical semiring $(\mathbb{R} \cup \{\infty\}, \min, +)$ for "cost to verify"; the Boolean semiring for bare support.

The instance we have already built is the smallest interesting one: the **confidence lattice** $\{\bot < \mathrm{Low} < \mathrm{Med} < \mathrm{High}\}$, a bounded distributive lattice with $\otimes = \wedge = \min$ and $\oplus = \vee = \max$. A bounded distributive lattice *is* an idempotent commutative semiring, so everything above applies verbatim.

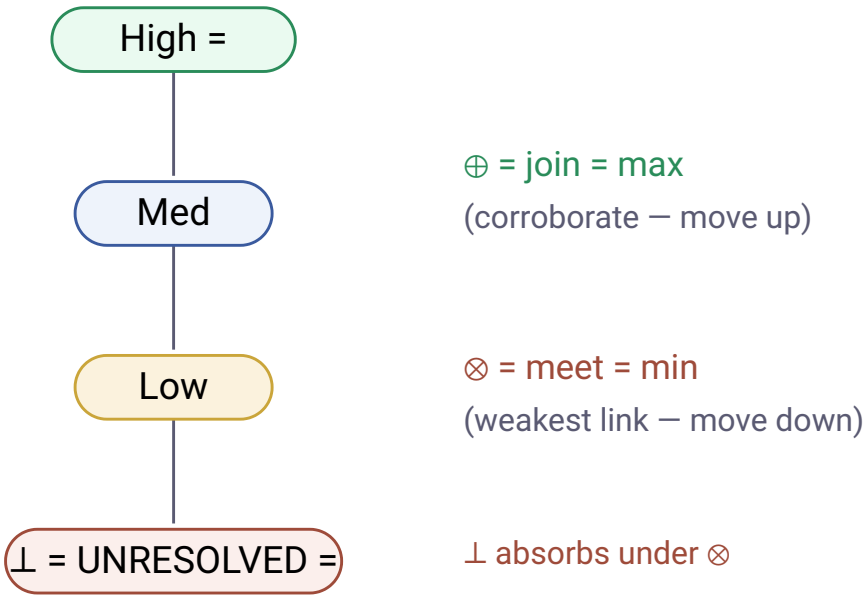

Figure 4 — The confidence lattice as a Hasse diagram. Conjunction is meet (drop to the weakest input); corroboration is join (rise to the best). $\bot$ at the bottom is absorbing under $\otimes$. This is the deterministic High/Med/Low signal, now seen as a four-element idempotent semiring.

4.3 · Fail-closed semantics

"Fail-closed" is a principle borrowed from security and safety engineering: when a result is missing, unverifiable, or uncertain, the system defaults to the safe, restrictive state — no usable value — rather than proceeding with a guess. The metaphor is a gate. **Fail-closed**: on failure the gate shuts and nothing passes. **Fail-open**: on failure the gate opens and everything passes. A lock that fails closed stays locked when the power dies; a firewall that fails closed denies all traffic when its rules crash; "default-deny" authorization is fail-closed. The two choices have opposite failure modes, which is the whole reason to pick one: a fail-closed system's mistakes are over-refusals — it returns nothing when it could have answered, a loss of throughput, not of correctness — while a fail-open system's mistakes are false accepts: it lets a wrong value through with authority, a loss of correctness.

In this algebra, fail-closed *is* the annihilator law of §4.1, $0 \otimes x = 0$, with `UNRESOLVED` as the zero. When a value's support is missing it resolves to `UNRESOLVED` rather than a plausible number; and because `UNRESOLVED` absorbs under $\otimes$, any derived value that conjunctively depends on it is itself

UNRESOLVED — the failure propagates closed across every hop, and (since homomorphisms preserve the zero) under every trust model. The deliberate asymmetry is what makes this safe: the system is *allowed* to return UNRESOLVED and is never penalized for it, and is penalized only for a confident value that turns out wrong. The cost of uncertainty is paid as "no answer, escalate to a human," never as a wrong answer that reaches a signature. This inverts the default behavior of most language-model systems, which fail *open* — when uncertain they emit a fluent guess; flipping that default is the point.

Figure 5 — The annihilator at work. One UNRESOLVED conjunct collapses the whole derivation, no matter how confident the other inputs are. The agent that owns the ratio cannot emit a confident leverage figure when its debt input never resolved.

4.4 · Verification is a second, orthogonal axis

Confidence answers "*how strong is the derivation, assuming the inputs hold?*" Verification answers a different question: "*does the value actually reproduce against the provenance it cites?*" — re-resolve the citation, re-match the quote. This is a decidable predicate $\mathrm{verify} : \mathrm{Claim}\ a \to \mathrm{Bool}$, independent of the semiring grade. A claim can be high-confidence yet fail verification, or verify yet be low-confidence. The acceptance policy that gates a signature is the conjunction of the two:

$$\mathrm{accept}(c) \iff \mathrm{verify}(c) \wedge h(\mathrm{prov}(c)) \geq \theta.$$

Tracking both axes at once is a **bilattice** in the sense of Belnap's four-valued logic [7]: one axis for how much you believe, one for how much is confirmed.

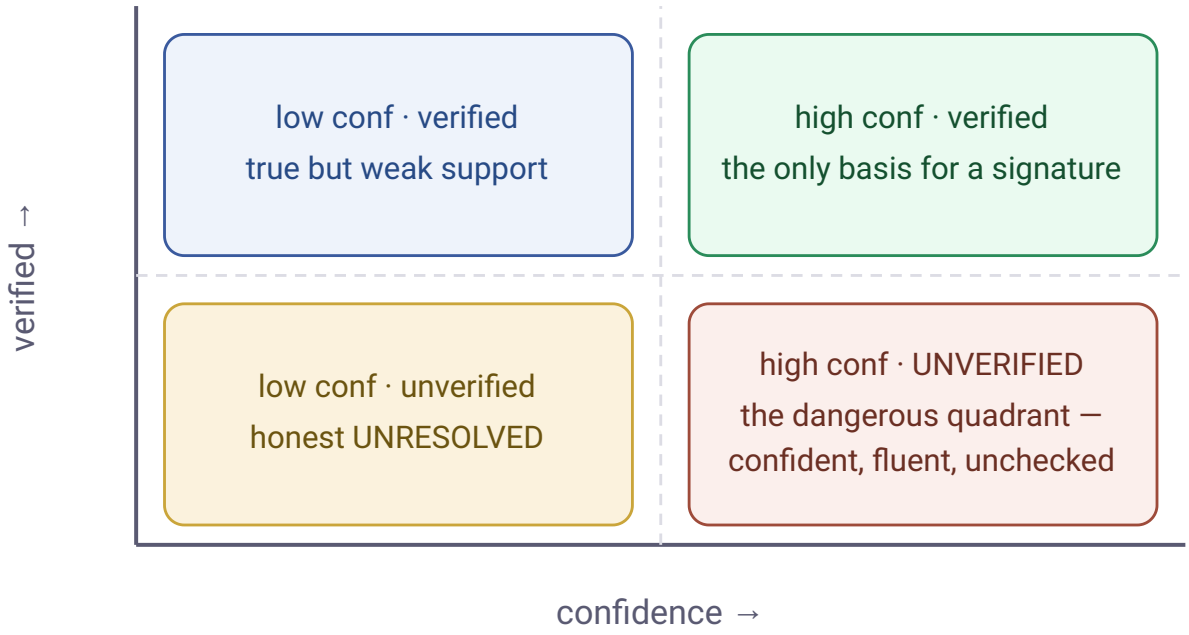

Figure 6 — The two axes are independent. Current agent swarms live in the lower-right danger quadrant: fluent and confident, unchecked. The algebra propagates the horizontal axis faithfully; the vertical axis is what `verify()` establishes, and only the upper-right quadrant should reach a signature.

4.5 · The properties, and what each one buys

PROPERTY	STATEMENT	WHAT IT BUYS AN AGENT NETWORK
Annihilation	$0 \otimes x = 0$, preserved by every homomorphism.	Fail-closed across arbitrarily many hops, under every trust model. The indispensable one.
Associativity + commutativity	$\otimes, \oplus$ are associative and commutative.	Order-independent fan-in: re-bracket sub-networks freely, parallelize, and memoize (resolve-once). Lost only for sequential value-dependent derivation.
Distributivity	$a \otimes (b \oplus c) = (a \otimes b) \oplus (a \otimes c)$.	Makes the homomorphism theorem hold; push a conjunct across corroborated alternatives consistently.
Homomorphic evaluation	Any $\nu : X \to K$ extends uniquely to $\hat{\nu} : \mathbb{N}[X] \to K$.	Compute lineage once; render any trust measure; swap the trust model without recomputing the network.

Monotonicity	More corroboration never lowers confidence; weakening an input never raises it.	No "evidence makes it worse" pathologies (in the positive fragment).
Idempotency (lattice K)	$a \oplus a = a$.	Re-receiving the *same* source does not inflate confidence. The choice of K **encodes your independence assumption**: idempotent $\min / \max$ is conservative; product/noisy-OR can reward corroboration but is fooled by correlated sources.

5 · The type layer CODE

The categorical statement of §3.2 — applicative = lax monoidal functor — is a programming type. All four mathematical readings converged on the same shape in Scala; here it is, lightly cleaned. The two names of §2 appear directly: `Axia[K, A]` is the graded value the algebra acts on, and `Claim[K, A]` is an Axia refined by verification.

```
// Confidence is a commutative semiring lifted with UNRESOLVED.
sealed trait Conf
case object Unresolved extends Conf                  // semiring 0: ⊗-annihilator, ⊕-identity
final case class Graded(p: Double) extends Conf      // p in [0,1]; Graded(1.0) = ⊗-identity ("cer

object Conf:
  def meet(a: Conf, b: Conf): Conf = (a, b) match    // ⊗  AND / weakest-link
    case (Unresolved, _) | (_, Unresolved) => Unresolved        // 0 ⊗ x = 0  — FAIL-CLOSED, struc
    case (Graded(x), Graded(y))            => Graded(x * y)      // product / weakest-link
  def join(a: Conf, b: Conf): Conf = (a, b) match    // ⊕  OR / corroboration
    case (Unresolved, y) => y
    case (x, Unresolved) => x
    case (Graded(x), Graded(y)) => Graded(x max y)               // or noisy-OR: x + y - x*y

// Provenance: a free commutative monoid over lineage atoms (union of derivations).
final case class Prov(atoms: Set[Lineage])
object Prov:
  given Monoid[Prov] = Monoid.instance(Prov(Set.empty), (a, b) => Prov(a.atoms | b.atoms))

// THE AXIA: a value carried with (provenance, grade). Value-agnostic — a fact, a cost, a trust.
final case class Axia[+A](value: Option[A], prov: Prov, conf: Conf)

object Axia:
  // CONJUNCTIVE — the Validation applicative: ACCUMULATES lineage, never short-circuits it.
```

```
  def and2[A, B, C](a: Axia[A], b: Axia[B])(f: (A, B) => C): Axia[C] =
    val c = Conf.meet(a.conf, b.conf)
    val v = if c == Unresolved then None else (a.value, b.value).mapN(f)
    Axia(v, Prov(a.prov.atoms | b.prov.atoms), c)               // lineage = union of ALL inputs

  // DISJUNCTIVE — corroboration (a Semigroup "combine up").
  def or[A](a: Axia[A], b: Axia[A]): Axia[A] =
    Axia(a.value.orElse(b.value), Prov(a.prov.atoms | b.prov.atoms), Conf.join(a.conf, b.conf))

// A CLAIM is an Axia refined by verification — an Axia that asserts, and can be checked.
type Claim[A] = Axia[A] Refined VerifiesAgainstProvenance       // verify(c) recomputes value vs
```

> **Why an applicative, not a monad.** Monadic `>>=` lets later structure depend on an earlier *value* and short-circuits on the first failure — discarding the lineage of the other conjuncts. The `Validation` applicative is deliberately *not* a monad, precisely so it *accumulates* every input's provenance [2]. That is exactly the Axia's requirement: a downstream agent must receive the full lineage of all conjuncts, for verification and blame-assignment. The "graded" qualifier means the $(\mathrm{Prov}, \mathrm{Conf})$ annotation is an effect grade produced by composition, never written by the payload [10]. When an agent genuinely chooses *whom to ask next* from a prior answer, that single edge is legitimately monadic — a justified escape hatch, not the default.

In Haskell the same object is `Axia a = Writer (Prov, Conf) (Maybe a)` exposed as an `Applicative` only, with `Claim` via a Liquid Haskell refinement on top. In the harness's Go (no higher-kinded types), the realization is narrower and deliberately so: an `Axia` struct carrying `value`, `prov`, and a `Conf` enum, with `And`/`Or` free functions that pattern-match `Unresolved` first — the annihilator becomes an explicit early return rather than a typeclass law, but the contract is identical.

---

6 · The workbench is the N = 1 instance

This is not a research program bolted onto unrelated work. The Credit & Loan Agreement Workbench is the single-agent specialization of exactly this object, which is the strongest evidence the abstraction is right.

CLAIM ALGEBRA	WORKBENCH REALIZATION
Provenance polynomial $\mathbb{N}[X]$	The defined-term DAG — terms as nodes, references as edges.
Evaluating the polynomial	Resolving a term by the topological walk.
Annihilator $0 \otimes x = 0$	"Fail closed to UNRESOLVED" on a missing reference or a cycle.

Confidence lattice K	The deterministic High/Med/Low signal.
Verification predicate	Native Citations / quote-then-verify against the source span.
Homomorphism theorem	"One source of truth → many renders" (abstract, Excel, memo, JSON).
Acyclicity precondition	The DAG's cycle-detection and hop-depth cap.

The leap from N = 1 to N agents is to make the nodes *agents* instead of *defined terms* — the same algebra, the same fail-closed guarantee, the same render-many-from-one. The concrete first step is to lift the harness's `LlmCall`/claim representation toward the `Claim[K, A]` shape, making the object explicit in code where it is currently implicit.

---

7 · A second instance: cost routing in an agent fulfillment network

Section 6 read the workbench as the $N = 1$ specialization at the confidence lattice. The same object at a different semiring is a network of fulfillment-center agents finding the cheapest way to move an item to a customer — the **cost-semiring sibling**. It is worth working through because it exercises the same three load-bearing moves (annihilator as fail-closed, verification as a second axis, the homomorphism) on a problem that looks nothing like clause extraction, and because it is the natural setting for many agents rather than one.

7.1 · The tropical specialization

Take K to be the **tropical (min-plus) semiring** $(\mathbb{R} \cup \{+\infty\},\ \min,\ +,\ +\infty,\ 0)$. Nodes are fulfillment centers; edges are lanes (center-to-center transfer, or center-to-customer last mile); an edge's weight is its cost. Conjunction $\otimes = +$ adds cost along a path; disjunction $\oplus = \min$ picks the cheapest among alternatives. The zero plays the same UNRESOLVED role it played for confidence, now with a telling **dual character**: as the $\oplus$-identity it is "*no route found yet*" (beaten by any real route), and as the $\otimes$-annihilator it is "*this leg is infeasible*" ($+\infty + x = +\infty$, poisoning any route through it). Optimal routing is the Kleene closure $a^* = 1 \oplus a \oplus a^2 \oplus \cdots$ of the tropical adjacency matrix — Bellman–Ford and Floyd–Warshall *are* this computation [14, 15]. It converges when there is no negative-cost cycle, a condition automatically met by non-negative shipping costs.

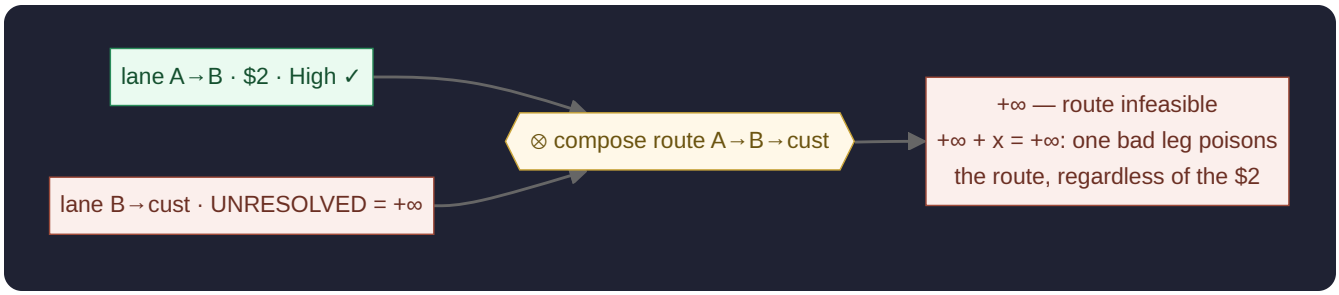

Figure 7 — The annihilator, in cost. A center that cannot confirm its onward lane advertises $+\infty$; $+\infty + x = +\infty$ is literally $0 \otimes x = 0$, so the route collapses no matter how cheap the other legs are. This is Figure 5's fail-closed law, specialized to the tropical semiring.

The "**ship direct, or forward through a closer center then ship**" decision is one $\oplus$ over $\otimes$-products:

$$\text{cost}(A \to \text{cust}) \;=\; [A \to \text{cust}] \;\oplus\; \big([A \to B] \otimes [B \to \text{cust}]\big) \;\oplus\; \cdots \;=\; \min\{\,9,\; 2{+}3,\; \ldots\},$$

so shipping direct at $\$9$ loses to forwarding through B at $\$2 + \$3 = \$5$. The three center "types" — distribution-only, direct-to-customer, hybrid — are not special cases in the algebra; they are a *derived* property of each node's incident edges (whether a center-to-customer edge is present), not a tag the math reads.

7.2 · Intelligent agents make the edge weight a claim

A scalar lane cost assumes the cost is known. An intelligent center *predicts* it — transit time, capacity, stockout risk — so the edge weight is not a number but a graded value with provenance and confidence: an `Axia[Cost]`. This is the §2 distinction earning its keep — a cost is not an assertion, so it is naturally an *Axia*, not a Claim; the `verifier` field below, which lets the receiver re-check the lane against evidence, is what lifts it to a `Claim[Cost]`. The construction needs naming precisely, because it is easy to mis-state:

- Under $\otimes$ **(compose a route)** it is a genuine product: add the cost (tropical $\otimes = +$) *and* meet the confidence (lattice $\otimes = \min$, the weakest leg) — componentwise.
- Under $\oplus$ **(choose among routes)** it is *not* the componentwise direct product. It is the **tropical-weight-with-output** construction familiar from weighted finite-state transducers [16]: the tropical cost *selects* the winning route, and the route, confidence, and provenance ride along as an output label — concatenated under $\otimes$, projected from the arg-min at each $\oplus$. It is a well-defined, associative $\oplus$ only once equal-cost ties resolve deterministically (equivalently, a **lexicographic** semiring with cost primary).

The route witness here is an *ordered* path — the free monoid under concatenation — a deliberate strengthening of §5's unordered `Prov(Set)` provenance. Order is load-bearing: the loop check in

§7.4 reads it.

7.3 · The message one center sends another

A center B advertises to a neighbor A, per destination, a cost-to-serve claim — which, written out, is a **path-vector advertisement**:

```
final case class ReachClaim(
  dest:     DestId,                                  // the QUESTION: which customer / region
  cost:     Cost | Unreachable,                      // tropical VALUE (Unreachable = +inf = UNRES
  route:    Vector[Leg],                             // PROVENANCE — an ORDERED path witness (free
  conf:     Conf,                                    // weakest-link confidence (lattice meet)
  verifier: EvidenceRef,                             // VERIFICATION handle — re-check the lane vs
  asOf:     Instant, ttl: Duration,                  // freshness (a min-lattice)
  basis:    Indicative | Reservable(CapacityToken))  // estimate vs committable hold (Boolean-AND)
```

A **relaxes** by composing its lane to B with B's claim: $\mathrm{cost} = \mathrm{lane}(A \to B) \otimes \mathrm{msg.cost}$ (tropical $+$, with $+\infty$ absorbing — fail-closed); $\mathrm{route} = (A \to B)$ prepended; $\mathrm{conf} = \mathrm{meet}$ (weakest link); freshness the $\min$; and basis reservable only if both legs are. A's own advertised cost is the $\oplus$ (min) over all neighbors, carrying the arg-min's label and keeping the runners-up as alternative lineage — which is exactly where this $\oplus$ begins to shade toward the Pareto $\oplus$ of the callout below. The message is thus a product of several little monoids at once — cost (tropical), confidence (lattice), route (free monoid), freshness (min-lattice), feasibility (Boolean) — which is why the $\oplus$ is selective rather than componentwise. And its `verifier` field is the §4.4 second axis: an advertised cost is a `Claim[Cost]` whose `verify()` re-resolves the lane against the agent's evidence, so a high-confidence-but-unverified quote sits squarely in the danger quadrant of Figure 6.

7.4 · Provenance buys loop-freedom — the sharpest point

Carrying the full `route` lineage, and discarding any advertisement whose route already contains the receiver, makes this a **path-vector** protocol — the AS_PATH loop check of BGP [17]. Path-vector routing is *inherently* loop-free and therefore has **no count-to-infinity**. The contrast is instructive: bare distance-vector routing (RIP) carries only a metric, so it leans on split-horizon and poison-reverse — which suppress only two-hop loops — plus a metric-as-infinity ceiling, and *still* suffers count-to-infinity on longer loops [18]. The headline is the gift: **the same provenance field that buys auditability buys loop-freedom.** And an `Unreachable` advertisement — $+\infty$, the annihilator — is an active **route withdrawal** (distinct from a stale entry timing out): a center refusing to guess a lane it cannot confirm, and the network declining to route through it.

> **What this does *not* give you for free.** It is tempting to say "compute the route once, then re-optimize for cost, time, or carbon by swapping the homomorphism." That is false, and a practitioner will object immediately: changing the objective changes the chosen lane. The homomorphism theorem (§3.3) re-evaluates a *fixed, un-collapsed* lineage polynomial under many valuations — but shortest-path *selection* is itself a fold under one cost valuation, and collapsing to the cost-min discards the alternatives, including the possibly-different min-time path. What is genuinely reused is the path-enumeration *structure*, not the per-objective minimization; each objective still runs its own min-fold, and materializing the full polynomial is exponential — which is the honest reason one re-runs Bellman–Ford per objective. True simultaneous multi-objective routing needs a **lexicographic** semiring (a total order — cost, ties broken by time) or a **Pareto / antichain** semiring (labels are *sets* of non-dominated cost vectors; $\oplus$ is non-dominated union, $\otimes$ is vector addition), the latter computing the frontier once but at a cost that is NP-hard with a possibly exponential frontier [19]. This is the cost-semiring analogue of §8's "aggregates need a semimodule" limit.

7.5 · The same object, a second time

CLAIM ALGEBRA	FULFILLMENT REALIZATION
Semiring K	Tropical $(\mathbb{R} \cup \{+\infty\},\ \min,\ +)$ — cost, not confidence.
Zero 0 = UNRESOLVED	$+\infty$ = unreachable: $\oplus$-identity "no route" and $\otimes$-annihilator "infeasible leg".
Annihilator = fail-closed	One infeasible leg poisons the whole route; a center withdraws rather than guesses.
Confidence lattice	The weakest-link confidence carried on an advertised cost.
Verification predicate	Re-check the advertised lane cost against the agent's evidence (rate card, tender).
Provenance polynomial	The ordered route witness — the path-vector that also gives loop-freedom.
Homomorphism (compute once)	Re-evaluate the route *lineage* under many objectives — *not* the collapsed arg-min (see callout).
Acyclicity precondition	No negative-cost cycle; loop-free by the path-vector check.

A network of center-agents converging on the optimum is **asynchronous distributed Bellman–Ford** [20] with `Claim[Cost]` as the messages — the document's Layer-2 claim network, run at

$K = \text{cost}$ instead of $K = \text{confidence}$. The workbench and the fulfillment network are the same object in two columns, which is the entire point of the abstraction.

---

8 · A third instance: adversarial games (Diplomacy)

The first two instances are cooperative — the workbench answers "is this true and cited?", the fulfillment network answers "what is the cheapest verified route?". The third is **adversarial**, and it is the most honest stress test, because it straddles the very boundary §9 draws. The game of Diplomacy [22] is the canonical AI-negotiation benchmark precisely because it has *no luck*: seven powers move simultaneously on a map of 1901 Europe, all variance lives in negotiation and in predicting others' moves, and an item of cheap talk between players carries no enforcement at all. It splits cleanly into two regimes.

8.1 · The substrate is the claim algebra proper

Board state, orders, and adjudication are a clean **verification-semiring** instance. Adjudication is a pure, deterministic function $\text{Orders} \times \text{Board} \to (\text{Board}, \text{Outcomes})$; the engine is the verification oracle. Every board fact is a `Claim[Verified, A]` — **no power can hallucinate the board**, and an illegal order hits the annihilator and is dropped to a Hold. Three claim types ride the wire: a `StateClaim` (a verified board fact), an `Order` (a Hold / Move / Support / Convoy commitment, legality-verified, adjudicated), and a `Deal` (a negotiation message — below).

8.2 · Negotiation leaves the clean semiring

A `Deal` — "I support your move to Galicia *if* you move there too" — is a **promise: a claim whose verification is deferred** to adjudication. At assertion time it is UNRESOLVED-as-fact, and the algebra structurally refuses to let cheap talk pose as a board fact. What a power can attach is a *trust* confidence — but, exactly as §9's "calibration is exogenous" and "no theory of adversarial sources" limits warn, that number does **not** come from provenance. It comes from **subjective logic** [8] — an opinion $(\text{belief}, \text{disbelief}, \text{uncertainty})$ over "will this power keep its word?" — updated by **reputation**: the verification *outcomes* of past promises (did their submitted orders match the pledge?). Subjective logic's fusion and discounting operators approximate, but are not, a clean commutative semiring — which is precisely the boundary the document draws, made concrete.

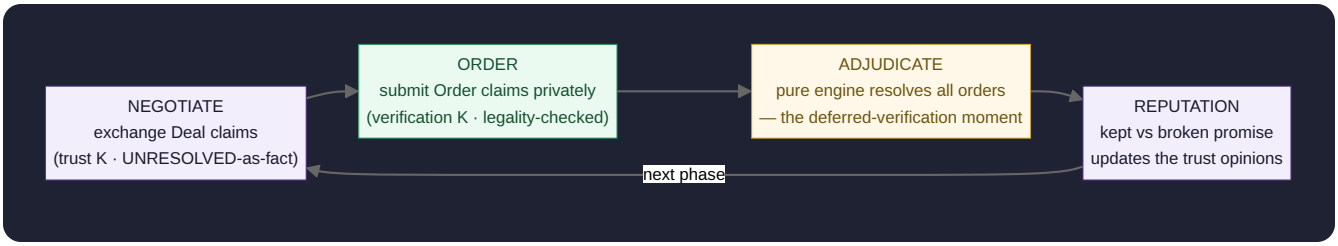

Figure 8 — Diplomacy as a deferred-verification loop. The substrate (Order → Adjudicate) is the verification-semiring claim algebra, fail-closed. The diplomacy (Negotiate → Reputation) leaves it: a promise is a claim verified only *after* the fact, and its confidence is a subjective-logic reputation, supplied by game theory — not by the algebra.

8.3 · Why it is the sharpest instance

A power *can* lie in negotiation — it is cheap talk, and deception is the game — but it *cannot* lie about the board or make an illegal move. The algebra **shrinks the attack surface to exactly the strategic layer where deception belongs**, and makes the whole game auditable and replayable: the transcript is a provenance DAG. So Diplomacy exercises both the object and its limit in one system — the substrate shows the algebra working, the diplomacy shows precisely where it stops and game theory takes over (Meta's **CICERO** reached human-level play by keeping its messages consistent with its planned moves [23]). The full design — the actor-model communication layer, the game loop, and the Scala realization — is the companion document `diplomacy-agent-game.html`.

---

9 · Where the algebra stops — the honest limits

Four independent mathematical analyses named the same boundaries. That convergence means they are real, and each is a layer you would add deliberately rather than a flaw to paper over.

- **Correlation blindness.** $\oplus$ assumes independence (or that dependence shows up as a shared leaf). Two agents echoing one upstream leak are scored as independent corroboration. The fix is to track token identity — probabilistic databases, where exact confidence is #P-hard in general [9] — or to move to Dempster–Shafer belief functions or Jøsang's subjective logic [8], which carry an explicit conflict/uncertainty mass. Idempotent K dodges double-counting an *identical* leaf but not a latent common cause.
- **Calibration is exogenous.** The algebra propagates leaf confidences faithfully but does not tell you the right leaf values; "High" means "high in the derivation order," not "true 90% of the time." Calibration is empirical, outside the type.
- **Semantic correctness is not verification.** `verify()` confirms the quote *reproduces*, not that it *answers the question asked*. A correctly-cited wrong clause verifies fine — the algebra gives

disciplined uncertainty propagation, not truth.

- **No conflict or non-monotonic retraction.** Positive semirings model only monotone composition; "A says 3.00%, B says 3.25%, both cited" collapses to the same $\bot$ as "unknown." Genuine contradiction and retraction need a bilattice [7] or semirings with monus. A network hits this immediately — an amendment that supersedes a base clause is a retraction in disguise.
- **Timeless and acyclic.** A source amendment forces recomputation rather than incremental update, and a cycle (A cites B cites A) silently re-inflates confidence unless an external check breaks it. Aggregates such as a leverage ratio additionally need a semimodule, not just a semiring [4].
- **Capacity contention (the routing instance's headline limit).** The §7 shortest-path routes *one* unit at a *fixed* cost. The moment two shipments contend for one lane's capacity, the problem becomes **min-cost flow** — a linear program or an auction — which is *not* a single semiring path [21]. The `Reservable(CapacityToken)` field only *marks* the seam (a reservation handle); it does not allocate. And static-snapshot costs versus time-varying ones is an MDP / online-routing problem, the cost-semiring face of the "timeless" limit above.

> **The contribution, stated precisely.** None of the mathematics is new — semirings (1950s), provenance semirings (2007), valuation algebras (1990), the validation applicative (2008). What is unbuilt is adopting a chosen evaluation semiring as the *typed wire format between LLM agents*: every inter-agent message a `Claim[K, A]`, fail-closed by the annihilator, trust-model-pluggable by the homomorphism, verifiable by the refinement. The value concentrates exactly where verification is the binding constraint — high-stakes, auditable, regulated work — and that is the wedge worth taking.

---

10 · References

1. T. J. Green, G. Karvounarakis, V. Tannen. *Provenance Semirings.* PODS 2007. — The provenance polynomial $\mathbb{N}[X]$ and the homomorphic-evaluation theorem.
2. C. McBride, R. Paterson. *Applicative Programming with Effects.* Journal of Functional Programming 18(1), 2008. — Applicative functors; the `Validation` (accumulating, non-monadic) pattern.
3. J. Cheney, L. Chiticariu, W.-C. Tan. *Provenance in Databases: Why, How, and Where.* Foundations and Trends in Databases, 2009. — The how ⊒ why ⊒ where provenance hierarchy.
4. Y. Amsterdamer, D. Deutch, V. Tannen. *Provenance for Aggregate Queries.* PODS 2011. — Why aggregates (SUM, AVG, ratios) need a semimodule over the semiring.
5. B. Fong, D. I. Spivak. *Seven Sketches in Compositionality: An Invitation to Applied Category Theory.* 2019. — Monoidal categories and string diagrams as a compositional language.

6. J. Baez, M. Stay. *Physics, Topology, Logic and Computation: A Rosetta Stone.* 2011. — Symmetric monoidal categories across domains.
7. N. D. Belnap. *A Useful Four-Valued Logic.* 1977; M. Ginsberg, *Multivalued Logics* (bilattices), 1988. — The two-axis (belief / verification) structure.
8. A. Jøsang. *Subjective Logic: A Formalism for Reasoning Under Uncertainty.* Springer, 2016. — Explicit uncertainty mass and source-dependence fusion.
9. N. Dalvi, D. Suciu. *Efficient Query Evaluation on Probabilistic Databases.* VLDB 2004. — #P-hardness of exact probabilistic confidence under correlation.
10. S. Katsumata. *Parametric Effect Monads and Semantics of Effect Systems.* POPL 2014; D. Orchard et al., graded monads/comonads. — Effects graded by an algebraic structure.
11. B. Day. *On Closed Categories of Functors.* 1970. — Lax monoidal functors (the categorical home of applicatives).
12. P. Selinger. *A Survey of Graphical Languages for Monoidal Categories.* 2011. — String-diagram semantics.
13. P. P. Shenoy, G. Shafer. *Axioms for Probability and Belief-Function Propagation.* 1990; J. Kohlas, *Information Algebras: Generic Structures for Inference*, Springer, 2003. — Valuation / information algebras and local computation.
14. M. Mohri. *Semiring Frameworks and Algorithms for Shortest-Distance Problems.* Journal of Automata, Languages and Combinatorics 7(3), 2002. — Algebraic shortest-distance over an arbitrary semiring; Bellman–Ford / Floyd–Warshall as semiring computation (§7.1).
15. D. J. Lehmann. *Algebraic Structures for Transitive Closure.* Theoretical Computer Science 4(1), 1977. — Closed semirings and the Kleene-star closure $a^* = 1 \oplus a \oplus a^2 \oplus \cdots$. See also Gondran & Minoux, *Graphs, Dioids and Semirings*, Springer, 2008, for the tropical dioid.
16. M. Mohri, F. Pereira, M. Riley. *Weighted Finite-State Transducer Algorithms: An Overview.* 2004 (and the OpenFst library). — The tropical-weight-with-output-label construction that correctly names the cost × route/confidence object (§7.2).
17. Y. Rekhter, T. Li, S. Hares (eds.). *RFC 4271: A Border Gateway Protocol 4 (BGP-4).* 2006. — The AS_PATH loop check (reject an advertisement whose path already contains the receiver) and explicit route withdrawal — path-vector loop-freedom (§7.4).
18. C. Hedrick. *RFC 1058: Routing Information Protocol.* 1988; G. Malkin, RFC 2453. — Distance-vector mechanics: split-horizon, poison-reverse, and the metric-16 = infinity ceiling — the contrast class where count-to-infinity is mitigated, not solved (§7.4).
19. L. Mandow, J. L. Pérez de la Cruz. *A New Approach to Multiobjective A* Search (NAMOA*).* IJCAI 2005; M. Ehrgott, *Multicriteria Optimization*, Springer, 2005. — The Pareto / antichain frontier of non-dominated label vectors for true multi-objective routing (§7 callout).
20. D. Bertsekas, J. Tsitsiklis. *Parallel and Distributed Computation: Numerical Methods.* 1989. — Convergence of asynchronous distributed Bellman–Ford (§7.5).
21. R. K. Ahuja, T. L. Magnanti, J. B. Orlin. *Network Flows: Theory, Algorithms, and Applications.* 1993. — Min-cost flow as the LP that capacity contention requires, beyond a single semiring path (§8 limit).
22. A. B. Calhamer. *The Rules of Diplomacy.* Avalon Hill / Hasbro, 1959 (4th ed. 2000). — The game; the deterministic, simultaneous-move, no-luck adjudication that makes it the negotiation benchmark (§8).
23. Meta FAIR Diplomacy Team (A. Bakhtin et al.). *Human-level play in the game of Diplomacy by combining language models with strategic reasoning.* Science 378(6624), 2022 (CICERO). — LLM dialogue kept

consistent with a strategic planner; honesty/intent-grounding of messages (§8).

Companion to the focused-harness actor-model design and the Credit & Loan Agreement Workbench technical design (§11, the model-to-human trust contract — of which this is the model-to-model generalization). Mathematics rendered with MathJax; flow diagrams with Mermaid; lattice, commuting square, and two-axis figures are inline SVG.